OBSERVATION :-

Program - 7

Vacuum World Problem

AIM: To implement a simple reflex Vacuum cleaner agent that cleans a two-room world (A and B)

Algorithm:

- Step-1: Start. Read starting location (A or B) and the status (clean/dirty) of both rooms.
- Step-2: percept = (current location, status of current room).
- Step-3: If the current room is dirty, perform suck and mark the room clean.
- Step-4: Else, if the location is A, move Right to B; if the location is B move left to A.
- Step-5: Count each action as cost 1.
- Step-6: Repeat steps 2 to 4 until both rooms are clean.
- Step-7: print final state and total cost. Stop.

Program:

Vacuum World - Simple Reflex Agent
(two rooms: A and B).

Percept = (location, status). The agent cleans if dirty, else moves to other room

```
def vacuum_agent(location, status):
    """Simple reflex agent: returns the action
       for the current percept."""
    if status == "Dirty":
        return "Suck"
    return "Right" if location == "A" else "Left"

def run_vacuum_world(location, rooms):
    cost = 0
    print(f"Initial State -> Location: {location}, Rooms: {rooms}\n")
    # Keep acting until both rooms are clean
    while "Dirty" in rooms.values():
        action = vacuum_agent(location, rooms[location])
        print(f"Location: {location} | Status: {rooms[location]} | Action: {action}")
        if action == "Suck":
            rooms[location] = "Clean"
        elif action == "Right":
            location = "B"
        else:
            location = "A"
        cost += 1
    print(f"\nFinal State -> Location: {location}, Rooms: {rooms}")
    print(f"Both rooms are clean. Total actions (cost) = {cost}")
```

```
if __name__ == "__main__":
    loc = input("Enter starting location (A/B): ").strip().upper()
    a = input("Is room A Dirty or clean? ").strip().capitalize()
    b = input("Is room B Dirty or clean? ").strip().capitalize()
    run_vacuum_world(loc, {"A": a, "B": b})
```

Output:

Enter starting location (A/B): A
Is Room A Dirty or clean? Dirty
Is Room B Dirty or Clean? Dirty
Initial state → location: A, Rooms: {'A': 'Dirty', 'B': 'Dirty'}

Location: A | Status: Dirty | Action: Suck
Location: B | Status: Clean | Action: Right
Location: B | Status: Dirty | Action: Suck

Final state → location: B, Rooms: {'A': 'Clean', 'B': 'Clean'}
Both rooms are clean. Total actions (work): 3

Program - 2

8 board puzzle.

AIM: To solve 8 puzzle using A* search with the Manhattan distance heuristic

Algorithm:

- Step 1: Start. Define the start state and the goal state (0 = blank tile)
- Step 2: Count inversions in the start state. If count is odd, print 'can't be solved' and stop
- Step 3: Heuristic h(n) = Sum of Manhattan distances of every tile from its goal position.
- Step 4: Put the start state in a priority queue with f(n) = g(n) + h(n), where g(n) = moves made so far.
- Step 5: Remove the state with the smallest f(n). If it's goal, go to step 7.
- Step 6: Mark it visited. Slide the blank Up, Down, Left, Right (where possible) to generate new states. Add unvisited to the queue with their f(n).
- Step 7: Repeat from step 5 until goal is found
- Step 8: print the sequence of moves and boards from start to goal.

Stop.

Program:

```
# 8 word problem - solved using A* search
  (Manhattan distance heuristic)
import heapq.

Goal
GOAL = (1,2,3,
        4,5,6,
        7,8,0)      # 0 represents the blank tile
START= (1,2,3,
        5,6,0,
        7,8,4)      # change this to try other
                      puzzles.

def manhattan(state):
    """ Sum of distances of each tile from
        its goal position. """
    total = 0.
    for i, tile in enumerate(state):
        if tile != 0:
            g = GOAL.index(tile)
            total += abs(i//3 - g//3) +
                     abs(i%3 - g%3).
    return total.

def is_solvable(state):
    """ A 3x3 puzzle is solvable only if
        the number of inversions is even. """
    tiles = [t for t in state if t != 0]
    inv = sum(1 for i in range(len(tiles))
              for j in range(i+1, len(tiles))
                  if tiles[i] > tiles[j]).
    return inv % 2 == 0
```

```
def neighbors(state):
    """Generate all states reachable by sliding one tile into the blank."""
    b = state.index(0)
    r, c = divmod(b, 3)
    for dr, dc, move in [(-1, 0, "Up"), (1, 0, "Down"), (0, -1, "Left"), (0, 1, "Right")]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            n = nr * 3 + nc
            s = list(state)
            s[b], s[n] = s[n], s[b]
            yield tuple(s), move

def solve(start):
    pq = [(manhattan(start), 0, start, [])]   # (f, g, state, path)
    visited = set()
    while pq:
        f, g, state, path = heapq.heappop(pq)
        if state == GOAL:
            return path
        if state in visited:
            continue
        visited.add(state)
        for nxt, move in neighbors(state):
            if nxt not in visited:
                heapq.heappush(pq, (g + 1 + manhattan(nxt), g + 1, nxt, path + [move]))
    return None
```

```
def show(state):
    for i in range(0,9,3):
        print(" ".join(str(x) if
            x else "_" for x in
            state[i:i+3]))
    print()

if __name__ == "__main__":
    if not is_solvable(START):
        print("This puzzle can't be
            solved.")
    else:
        print("Start State:")
        show(START)
        solution = solve(START)
        for step, (move, state) in
            enumerate(solution, 1):
            print(f"Step {step}: Move blank
                {move}")
            show(state)
        print(f"Goal reached in {len(solution)}
            moves.")
```

Outputs:

```
Start State:
1 2 3
5 6 _
7 8 4

Step 1: Move blank left
1 2 3
5 _ 6
7 8 4
```

Step 2: Move blank Left

1 2 3
_ 5 6
7 8 4

Step 3: Move blank Down

1 2 3
7 5 6
_ 8 4

Step-4: Move blank Right

1 2 3
7 5 6
8 _ 4

Step 5: Move blank Right

1 2 3
7 5 6
8 4 _

Step 6: Move blank UP

1 2 3
7 5 _
8 4 6

Step 7: Move blank Left

1 2 3
7 _ 5
8 4 6

Step 8: Move blank Down

1 2 3
7 4 5
8 _ 6

Step 9: Move blank left

1 2 3
7 4 8
_ 6 5

Step 10: Move blank Up.

1 2 3
_ 4 5
7 8 6

Step 11: Move blank Right:

1 2 3
4 _ 5
7 8 6

Step 12: Move blank Right

1 2 3
4 5 _
7 8 6

Step 13: Move blank Down

1 2 3
4 8 6
7 _ _

Goal reached in 13 moves.

Program - 3.

8 Queens problem

Aim: To place 8 Queens on 8*8 chessboard so that no two queens attack each other using backtracking

Algorithm:

- Step 1: Start: Use board[row] = column of queen placed in the row
- Step 2: Begin with row 0.
- Step 3: For each column 0 to 7 in current row, check if its safe: no queen in the same column and none of the same diagonal (|col| different |row difference|).
- Step 4: If safe, place the queen and recursively try the next row
- Step 5: If all 8 rows are filled, save the board as a solution
- Step 6: If no column is safe in a row, backtrack: remove the previous queen and try its next column
- Step 7: Continue until all possibilities are explored. print number of solutions and the first one. Stop.

Program:

```
# 8 Queen problem - solved using Backtracking
N=8

def is_safe(board, row, col):
    """ board[r] = column of the queen in
        row r. Checks column and diagonal
        clashes """
    for r in range(row):
        if board[r] == col or abs(board
                [r]-col) == abs(r-row):
            return False
    return True

def solve(board, row, solutions):
    if row == N:                    # all 8 Queens placed
        solutions.append(board[:])
        return
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            solve(board, row+1, solutions)
            board[row] = -1         # backtrack

def print_board(board):
    for r in range(N):
        print(" ".join("Q" if board[r]
            == c else "." for c in range(N)))

if __name__ == "__main__":
    solutions = []
    solve([-1]*N, 0, solutions)
    print(f"Total solutions for {N} Queens: {len(solutions)}")
    print("First solution:")
    print_board(solutions[0])
    print("\n Queen positions (row, column):", [(r+1, c+1)
        for r,c in enumerate(solutions[0])])
```

Output:

Total solutions for 8-Queens: 92

First solution:

Q . . . . . .
. . . . Q . . .
. . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

Queen positions (row, column):

[(1,1), (2,5), (3,8), (4,6), (5,3), (6,7), (7,2), (8,4)]

Program - 4

Water Jug problem.

Aim. To measure a required quantity of water using two jugs of given capacities using BFS

Algorithm:

- Step 1: Start. Read capacities of jug x and jug Y and the target amount
- Step 2: A state is (x, y) = water in X and Y. Put (0, 0) in a queue and mark it visited
- Step 3: Remove the first state from the queue. If x or y equals the target, go to step 6
- Step 4: Generate next states: fill X, fill Y, empty X, empty Y, pour X into Y, pour Y into X.
- Step - 5: Add each unvisited new state (with its path) to the queue and mark it visited. Go to step 3.
- Step 6: print path of state from (0, 0) to the target. If the queue becomes empty, print 'No solution'. Stop.

Program:

```
# water Jug Problem - solved using BFS.
from collections import deque

def water_Jug (cap_x, cap_y, target):
    start = (0, 0)                    # (water in jug x, water in jug y)
    queue = deque ([start, [start]])
    visited = {start}
    while queue:
        (x, y), path = queue.popleft()
        if x == target or y == target:
            return path
        pour_xy = min (x, cap_y - y)      # amount to pour x -> y
        pour_yx = min (y, cap_x - x)      # amount to pour y -> x
        next_states = [
            (cap_x, y),                   # fill x
            (x, cap_y),                   # fill y
            (0, y),                       # empty x
            (x, 0),                       # empty y
            (x - pour_xy, y + pour_xy),   # pour x into y
            (x + pour_yx, y - pour_yx),   # pour y into x
        ]
        for s in next_states:
            if s not in visited:
                visited.add(s)
                queue.append((s, path + [s]))
    return None
if __name__ == "__main__":
    cap_x = int(input("Enter capacity of jug x: "))
    cap_y = int(input("Enter capacity of jug y: "))
    target = int(input("Enter target amount of water: "))
    path = water_Jug(cap_x, cap_y, target)
    if path is None:
        print("No solution exists.")
```

else:

print("\n Steps (X, Y): ")

for i, state in enumerate(path):

print(f" Step {i}: {state}")

print(f"\n Target reached in {len(path)-1} steps.")

Output.

Enter Capacity of Jug X: 4

Enter Capacity of Jug Y: 3

Enter target amt of water: 2

Steps (X, Y):

Step 0: (0, 0)

Step 1: (0, 3)

Step 2: (3, 0)

Step 3: (3, 3)

Step 4: (4, 2)

~~Step 5: (0, 2)~~

~~Step~~

Target reached in 4 steps

classmate
Date
Page

Program 5-

Missionaries & Cannibals problem

Aim: To move 3 missionaries and 3 cannibals across a river with a 2-person boat so that the missionaries are never outnumbered, using BFS

Algorithm:

- Step 1: Start. A state is (M on left, C on left, boat side). Start = (3,3,left), Goal = (0,0,Right).
- Step 2: put start state in a queue and mark it visited.
- Step 3: Remove the first state. If it is the goal, go to step 6.
- Step 4: Try each boat load: 1M, 2M, 1C, 2C, or 1M+1C, moving from the boat's current side to the other side.
- Step 5: Accept the new state only if it is valid (counts within 0 to 3, and on each bank missionaries aren't outnumbered by cannibals unless there are no missionaries) and not visited. Add it to the queue. Go to step 3.
- Step 6: print the path of crossing goals from start to goal. Stop.

Program:

```
# Missionaries and cannibals problem - solved using (BFS)
# State = (missionaries on left, cannibals on left, boat side: 'L' or 'R')
from collections import deque

TOTAL_M, TOTAL_C, BOAT = 3, 3, 2
MOVES = [(1,0), (2,0), (0,1), (0,2), (1,1)]
                                # missionaries, cannibals in a boat

def is_valid(m, c):
    """Missionaries must never be outnumbered by cannibals on either bank."""
    if not (0 <= m <= TOTAL_M and 0 <= c <= TOTAL_C):
        return False
    rm, rc = TOTAL_M - m, TOTAL_C - c
    if m > 0 and c > m:
        return False
    if rm > 0 and rc > rm:
        return False
    return True

def solve():
    start, goal = (TOTAL_M, TOTAL_C, "L"), (0, 0, "R")
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        (m, c, side), path = queue.popleft()
        if (m, c, side) == goal:
            return path
        for dm, dc in MOVES:
            if side == "L":
                nm, nc, nside = m - dm, c - dc, "R"
```

else:

nm, nc, nside = m-dm, c-dc, "L"

state = (nm, nc, nside)

if 0 <= nm <= M and 0 <= nc <= C and nm >= nc or nm == 0 and state not in visited:

visited.add(state)

direction = "→" if side == "L" else "←"

action = f"Move {dm}M {dc}C {direction}"

queue.append((state, path + [(state, action)]))

return None

if __name__ == "__main__":

path = solve()

print(f"{'Step':<5} {'Action':<18} {'Left bank (M,C)':<18} {'Right bank (M,C)':<18} Boat")

for i, ((m, c, side), action) in enumerate(path):

left = f"({m},{c})"

right = f"({M-m},{C-c})"

print(f"{i:<5} {action:<18} {left:<18} {right:<18} {'Left' if side=='L' else 'Right'}")

print(f"\nAll people crossed in {len(path)-1} crossings.")

Output:

~~Step Action Left bank (M,C)~~

Step	Action	Left Bank (M,C)	Right Bank (M,C)	Boat
0	Start	(3,3)	(0,0)	Left
1	Move 0M 2C →	(3,1)	(0,2)	Right
2	Move 0M 1C ←	(3,2)	(0,1)	Left
3	Move 0M 2C →	(3,0)	(0,3)	Right
4	Move 0M 1C ←	(3,1)	(0,2)	Left
5	Move 2M 0C →	(1,1)	(2,2)	Right
6	Move 1M 1C ←	(2,2)	(1,1)	Left
7	Move 2M 0C →	(0,2)	(3,1)	Right
8	Move 0M 1C ←	(0,3)	(3,0)	Left
9	Move 0M 2C →	(0,1)	(3,2)	Right
10	Move 1M 0C ←	(1,1)	(2,2)	Left
11	Move 1M 1C →	(0,0)	(3,3)	Right

All people crossed in 11 crossings